

Phase 3 — Name the fault, and show the working

What Phase 3 is, what is finished, what it found when it was pointed at the real pump, and what still has to change. Every number here is read from the live database, from git, from the rule table, or from the test suites at the moment this was built — 28 September 2026, 15:45.

Phase 3 is code-complete and merged. All ten tickets are built, tested and pushed. The platform can now describe what a signal is doing, grade a machine against the ISO standard, score its health from eight separate inputs instead of a three-entry lookup, and say which plot would prove any of the 15 faults it knows about.

And it currently names no fault on the only real machine. That is not a bug in this phase and it is the most important line in this document. At 0.278 s per capture, 3.60 Hz per spectrum line, the diagnostic frequencies a bearing fault lives at cannot be separated from ordinary shaft harmonics — they land on the same spectrum line. The fault engine is correct and it is being fed a picture too blurred to read. Lengthening the capture is what turns it on, and that is a gateway setting, not a code change.

The platform says so, rather than reporting a healthy machine. This is the whole point of the phase. Every screen that could show "nothing found" instead shows why nothing could be found.

1,910 automated tests across the suites (1152 backend, 22 gateway, 364 vibrationbot, 372 frontend), all run while building this document: **FAILING: backend**. 4 commits since Phase 2, 28 files changed, 5,567 lines added and 7 removed.

What Phase 3 was for, in one paragraph

Phase 1 made the measurements trustworthy and learned what normal looks like. Phase 2 scored how unusual each reading is and turned repeated unusual readings into alarms. Neither of them ever says *what is wrong* — an alarm says "this feature is behaving oddly", which tells a maintenance engineer to go and look but not what to look for. Phase 3 is the step that puts a name on it: unbalance, misalignment, a bearing outer-race defect. And, because a name is a claim somebody will act on, it is also the step that has to show its working.

Everything in this phase follows one rule, and it is the same rule as the two phases before it: **an answer the platform could not work out is shown as an absence with a reason, never as a neutral value**. A machine nobody has measured does not score 100 out of 100. A spectrum that cannot see a bearing fault does not report a healthy bearing. "We could not tell" and "nothing is wrong" look identical on a screen and mean opposite things, and every one of the four problems found late in this phase was a case of the first quietly becoming the second.

The ten tickets, and what each one actually does

Ticket	What it does, in plain terms	State
VIK-050	Works out whether the spectrum can tell two diagnostic frequencies apart at all. If it cannot, that is recorded as the instrument's limit rather than the machine's condition.	Done
VIK-051	Describes what the signal is doing before anything decides what it means — a harmonic series, sidebands, impacting. Each observation carries the numbers behind it.	Done
VIK-052	Gathers what is known about the machine: bearing type, vane count, speed. A detail nobody recorded switches off the rule that needs it instead of being guessed.	Done

Ticket	What it does, in plain terms	State
VIK-053	Runs the 15 fault rules against a capture and ranks what fits.	Done
VIK-054	Grades how far along a fault is. A stage climbs one step at a time and never reaches critical on a single reading.	Done
VIK-055	Replaces a fake health score (normal 100, warning 60, critical 20) with one built from eight real inputs.	Done
VIK-056	Gives the ISO 10816-3 zone. Where the machine record is ambiguous, returns every table that could apply.	Done
VIK-057	Stores a finding as a record with its evidence attached.	Done
VIK-058	A finding evolves rather than being re-created, so "developing for nine days" is answerable.	Done
VIK-059	Maps each fault to the plots that would prove it — 34 rows covering all 15 rules.	Done

Checked against the requirement document, not against memory

The ten tickets above being finished is not the same claim as the requirement being met, so Phase 3 was audited line by line against section 9.2 of the requirement document, which lists nine things that must appear beside every suspected fault. **Four were missing** — and they were the four a maintenance engineer reads first: which family the fault belongs to, whether it is getting worse, what to do about it, and whether it needs a shutdown or only an inspection. All four are now built. The table below is read from the database schema as this document is generated, so it is a check rather than a claim.

Requirement 9.2 asks for	Present
Fault name	yes
Fault family	yes
Severity	yes
Confidence	yes
Evidence	yes
Related plots	yes
Trend direction	yes
Recommended next action	yes
Shutdown or inspection	yes

The lesson is the same one the four runtime problems taught, in a different form: a ticket marked done records that somebody built what the ticket described, not that the requirement behind it is satisfied. Requirement 14 asked twice for the two that were missing — "How urgent is it?" and "What should the analyst do next?" — and neither had an answer anywhere in the platform.

One decision inside that work is worth stating on its own. How urgent a fault is now depends on how well the machine can actually be seen, not only on how bad the fault looks. A finding at the top stage cannot recommend stopping a machine if the capture failed its own quality checks, or if the spectrum could not separate the frequencies the diagnosis rests on. The platform says what it would have recommended and what held it back, so the gap gets closed rather than overridden. On this pump every finding is currently held at "look

at it when convenient" for exactly that reason.

The one thing that stops Phase 3 working, in numbers

This is the finding of the phase, and it is worth reading even if nothing else here is. It is not an opinion and it is not a software defect — it is arithmetic about the capture settings.

A spectrum is a list of how much vibration sits at each frequency, and it can only resolve detail down to one divided by the length of the recording. The gateway records **13,888 samples at 50 kHz, which is 0.278 of a second** — so its spectrum has a line every **3.60 Hz** and nothing whatever can be seen between them. The pump turns 24.67 times a second, so one line is **0.146 of a turn of the shaft**.

A fault is identified by finding energy at one frequency and not at another one nearby. So what matters is not where a single frequency sits, but whether a *pair* of them can be told apart. Two frequencies 1.58 Hz apart are the same line when a line is 3.60 Hz wide. The table below is the platform's own verdict, read from the function the pipeline calls — it is not a second opinion worked out for this document.

These two cannot be told apart	Gap between them	As a fraction of one spectrum line
BSF vs 2x shaft harmonic	0.49 Hz	0.14 of a line — same place
BPFO vs 3x shaft harmonic	1.58 Hz	0.44 of a line — same place
BPFI vs vane or blade pass	1.58 Hz	0.44 of a line — same place
FTF vs sub-synchronous rub	2.89 Hz	0.80 of a line — same place

Two frequencies need to be about 3 lines apart before a peak at one can be told from a peak at the other, because a single tone smears across neighbouring lines rather than landing cleanly on one.

Look at what each bearing frequency is confused with: BSF against 2x shaft harmonic; BPFO against 3x shaft harmonic; BPFI against vane or blade pass. Every one of those is something a perfectly healthy pump produces all day — shaft harmonics come from ordinary rotation, and vane pass is just the impellers going past. The two sit closer together than a single spectrum line, so **a damaged bearing and an ordinary machine produce the same picture**, and no amount of rule logic recovers the difference. The remaining pair, FTF against sub-synchronous rub, is one bearing symptom against another fault rather than against a healthy one.

Measured on the live data rather than argued from theory: across all eight channels of the most recent capture, the engine produces **zero** fault hypotheses even with the score threshold dropped to nothing. 5 of the 15 rules are switched off before they start because they need energy at one turn of the shaft and there is no peak there on seven of the eight channels.

What would fix it. Resolution is one divided by the recording length, so the recording has to get longer. The platform states the figure itself: separating every pair above needs about **6.1 seconds** of recording against the 0.278 being taken now — roughly **22 times longer**. This is a setting on the gateway. It costs storage and nothing else.

The second hardware problem, carried over from Phase 1

A separate limit, and the one Phase 1 already flagged. The accelerometers are set up for a range far larger than this pump actually produces, so the signal arrives using a tiny fraction of the available precision. In the most recent capture the eight channels contain between **12 and 73 distinct values** across the whole recording. Most of what the spectrum contains is the converter rounding up and down, not the machine.

This is what made the ISO grading return "Zone A — typical of newly commissioned machines" on the real pump, from a reading of 0.027 mm/s where a running pump reads one to three. The platform now refuses to grade a capture this coarse and says why, but refusing is damage control: the underlying signal still needs fixing at the PLC.

Four problems found by running it, not by reasoning about it

All four were invisible in the tests and obvious within minutes of pointing the code at the real gateway and the real database. Each one produced a confident, plausible, wrong answer rather than an error, which is the failure mode this programme is most exposed to. All four are fixed, and each now has a test that fails if it comes back.

What it did	Why it mattered
The symptom detection never ran at all, on any capture, ever.	It sat behind a check for shaft speed that this gateway does not always pass. The feature was written precisely for captures where no fault can be named — so the one situation it existed for was the one it could not run in. Symptoms are now recorded for every channel of every capture; the live database holds 18 observations across 8 channels.
ISO grading called the pump "typical of newly commissioned machines".	From a velocity two orders of magnitude below any running pump, because there was almost no signal to measure. The best zone in the standard and a machine the instrument cannot see produce the same number. Somebody would have read it as a clean bill of health.
It reported the 713th and 877th harmonics of running speed.	The tolerance is a fraction of a turn, so far up the spectrum almost any peak satisfies it by coincidence. Scattered numbers are not a harmonic series. It now reports the longest unbroken run instead.
It printed "243378% of the envelope".	An energy divided by an amplitude — the units do not cancel, so the figure was not a proportion of anything. It is now a share of the four bearing bands, which is also the question being asked.

A fifth was caught by the tests rather than by running it, and is worth recording for the same reason: the first draft of the fault-to-plot table invented nine fault names the engine cannot produce — `looseness` where the rule is called `mechanical_looseness`, and eight more. Every one would have shown an empty panel rather than an error. A test now checks the table against the rule list.

The health score, and why it is different now

What it used to be: normal is 100, warning is 60, critical is 20. That is the status word written in a different font. It moves in steps of forty, it cannot tell a machine that went to warning ten minutes ago from one that has been there six weeks, and its 100 is indistinguishable from the 100 of a machine nobody has ever taken a reading from.

It is now built from eight inputs — the severity of any named fault, how unusual the readings are against the machine's own learned baseline, the symptoms observed in the signal, whether it is getting worse, standing alarms, how well the machine can be seen at all, how critical it is, and what it has done in the past. Three decisions in how those combine are worth stating:

Decision	Why
Poor data lowers the ceiling , never the score.	A machine with no learned baseline cannot be certified at 100 — not because anything is wrong with it, but because nobody has looked hard enough to say. It tops out at 72 and the screen shows "/ 72" next to the number, so the shortfall is not blamed on the machine.

Decision	Why
One severe fault outranks four mild concerns.	Adding penalties up gets this backwards, and the mistake is invisible — both machines just end up with a number. They combine as independent concerns instead, so the worst one leads.
How critical the machine is changes the priority, not the score.	A spare pump and a boiler feed pump in identical mechanical condition are in identical mechanical condition. Letting criticality move the health figure makes one number answer two questions and check against neither.

Every point deducted carries the sentence that produced it. Hovering the score on the dashboard shows the largest single contribution and what it cost. A score nobody can take apart is the lookup table again, with more decimal places.

What is visible on the system now

Where	What changed
Dashboard — machine cards	A real health score where most machines previously showed nothing, written as "Health 12.4 / 72 · critical". The second number is the ceiling and only appears when it is below 100. Hovering gives the reason in a sentence.
Dashboard — machines with no data	"Not scored" in italics instead of an empty space, with a tooltip saying no capture has ever been taken. A blank cell reads as fine; this does not.
Dashboard — fleet average	Averages real scores and skips machines that cannot be scored, rather than counting them as healthy.
/api/v1/diagnosis/symptoms	What the signal is doing, per channel, with the numbers behind each observation. Currently the most useful of the five — it returns 18 observations on the latest capture.
/api/v1/diagnosis/findings	Named faults with their evidence and the plots that would prove them. Returns empty on this machine, with the reason attached.
/api/v1/diagnosis/health	The full health breakdown, every contribution itemised.
/api/v1/diagnosis/iso	The ISO zone, or every zone it could be and what is missing. Currently refuses on this machine because the signal is at the converter floor.
/api/v1/diagnosis/plot-evidence	The fault-to-plot table: 34 rows over 15 faults, each saying what to look for and why that plot.

What is in the database right now

Table	Rows	What it means
Captures ingested	162	Every recording the gateway has sent.
Feature scores	36,750	Individual readings scored against a learned baseline.
Alarms standing	2	Past all four conditions: repeated, rising, steady speed, trustworthy capture.
Symptom rows	8	One per channel per capture. 18 observations in total.
Fault findings	0	Zero, for the resolution reason above — not because the machine is healthy.
Fault-to-plot rows	34	Covering all 15 rules the engine knows.

Table	Rows	What it means
Machines / sensors	2 / 1	One pump is instrumented and sending data.
Database version	040	Migrations applied, in order, with a manifest guarding the chain.

What the platform can currently say about the signal

Five checks exist. Three of them are written in turns of the shaft and need a running speed; two do not. Every row records how many could run, so that an empty result is never mistaken for a quiet machine. On this capture a running speed was available, so all five ran on every channel — the observations below are what they found, not what they were prevented from looking for.

Channel	Observations found	Checks that could run
Channel 0	1	5 of 5
Channel 1	2	5 of 5
Channel 2	4	5 of 5
Channel 3	1	5 of 5
Channel 4	2	5 of 5
Channel 5	2	5 of 5
Channel 6	2	5 of 5
Channel 7	4	5 of 5

How much of this is actually checked

Suite	Tests	Result
backend	1,152	FAILED
gateway	22	passed
vibrationbot	364	passed
frontend	372	passed
Total	1,910	see above

Tests were also tested. A test that passes whether or not the code is correct is worse than no test, because it is a claim of cover that is not true. Twelve deliberate faults were introduced one at a time — removing the guard that stops a harmonic series being reported as sidebands, making poor data lower the score instead of the ceiling, letting criticality move the condition — and the suite was checked for whether it noticed. All twelve were caught.

Two were not caught on the first attempt, and both are worth recording. One was covered by a second, independent guard, so removing the first changed no behaviour — safe, but it meant the guard under test had no test of its own. The other slipped through because the test signal let a different rule reject it before the guard being tested was consulted. Both now have signals built specifically to isolate them.

What happens next

Whoever owns the gateway — and this is the one that matters. The capture length needs to go up from 0.278 of a second. Roughly **two seconds** separates the outer-race and inner-race frequencies, which are the two that matter most; **about 6.1 seconds** separates all four, including the ball-spin frequency, which is the closest pair of all. Until that happens the fault engine cannot name anything on this pump however correct it is. This is a configuration change, and it costs storage and nothing else.

Whoever owns the PLC. The accelerometer sensitivity question from Phase 1 is still open and still limiting every amplitude the platform reports. It is why the ISO grading currently refuses to answer.

Atharva. Phase 3 is finished and pushed. Two things are worth doing before Phase 4 regardless of the hardware: a gateway silence alarm — nothing anywhere in the codebase notices when the gateway stops sending, and it has gone quiet for days at a time before — and wiring the symptoms onto a screen, since they are currently the only thing the platform can say about this machine and they are reachable only through the API.

Laxman. Nothing in this phase blocks his work. The one thing to carry forward is the pattern behind all four late findings: when a feature is added behind a condition, check that the condition is ever true on the real gateway. Three of the four problems above were features that worked perfectly and never ran.

Built from the repository at commit e240068, the live database, the rule table, and the test suites as they ran at 28 September 2026, 15:45. The script that produced it is `backend/scripts/build_phase3_report.py`.